



۸۱۰۱۰۱۴۹۲

سیستم‌های یادگیری ماشین توزیع‌شده
نام و نام خانوادگی: علیرضا کریمی



تمرین دستی ۱

۱

اسلاید ۲ صفحه ۳ [۱]

چه در *tranining* و چه در *inference* و محاسبات عددی و شبیه سازی کار های ML به جبر خطی و نرم افزار های مربوط به آن نیاز داریم. بیشترین حافظه و زمان در ML ، صرف کارهای جبر خطی می‌شود. مثل ضرب ماتریسی و عملیات های *multiply and add* و غیره.

به دو روش میتوان زمان و *resource* مورد نیاز را محاسبه کرد:

۱. آنالیز کردن پیچیدگی های تئوری و محاسبه و تخمین طبق آن

۲. روش های *profiling*: روشی است برای محاسبه *resource* مصرفی هر کدام از *subroutine* های مختلف برنامه در زمان اجرا و مانیتور کردن منابع سیستم

۲

اسلاید ۲ صفحه ۱۰ [۱]

بهتر است تا جایی که امکان دارد از لول های بالاتر تسفاده کنیم چون تمام فرصت های بهینه سازی بیشتری نسبت به حالت عادی مدنظر قرار گرفته. مثلاً در لول ۳، بهینه سازی مربوطه به حالت ضرب ماتریکس در ماتریکس در نظر گرفته شده اما در لول ۲ باید یک جمع بزنیم روی حاصل یک سری ضرب ماتریکس در وکتور. پس به جای یک *call*، چندین *call* داریم و در نهایت آنها را *aggregate* میکنیم. خود همین موضوع امکان بهینه سازی های سطح بالاتر را از ما میگیرد. حتی ممکن است این *call* های متعدد، دسترسی *io* به حافظه را بیشتر از حد معمول کند که این خودش کند کننده است.

کلاً هم استاد سر کلاس گفتند، اگر یک لوپ را خودمان بنویسیم تا وقتی که همان را با توابع سطح بالاتر *BLAS* پیاده سازی کنیم، سرعتش به مراتب بیشتر میشود. نسبت به لوپ عادی، به خاطر یک سری بهینه سازی ها که انجام میدهد.

اسلاید ۲ صفحه ۱۴ [۱]

درمورد *blocking*:

به جای ضرب دو ماتریس بزرگ به صورت یکجا، ماتریس‌ها را به زیرماتریس‌های کوچک‌تر (*Block*) تقسیم می‌کنیم که سایز آن‌ها طوری انتخاب شده که در کش جا شوند. چون ماتریس‌ها به صورت یک جا نمیتوانند در کش باشند و کش محدودیت دارد، این کار را کردیم (اگر از حافظه رم استفاده کنیم، *overhead* بخش *communication* بسیار بیشتر خواهد بود)

حال مشخص کردن اندازه *block size* بهینه، بسته به معماری پردازنده و اندازه کش و یا مموری دارد. و سوال ساده ای نیست

روش *blocking* باعث افزایش *performance* محاسبات مربوط به ضرب ماتریکس ماتریکس یا ماتریکس وکتور می‌شود.

۳

اسلاید ۲ صفحه ۲۴ [۱]

ترتیب ایندکس‌ها در ماتریکس‌های *row - major* به این صورت است:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

اما ترتیب ایندکس ذخیره سازی ماتریکس در حالت *column - major* به این گونه است:

$$\begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix}$$

پس باید محاسبات ضرب ماتریسی که انجام می‌دهیم، همه ماتریس منطبق بر فقط یکی از این حالت‌ها باشند. در غیر اینصورت باید ترانهاده یکی از ماتریس‌ها را حساب کنیم.

اسلاید ۲ صفحه ۲۴، ۲۵، ۸ [۱]

نکته این است که بسیاری از نرم افزارهای محاسبات عددی مربوط به کارهای جبر خطی، کتابخانه‌های مورد نیازش در زبان *FORTRAN* پیاده سازی شده، و حتی بعضی از پیاده سازی‌های مختلف *BLAS* مثل *ATLAS* و *Eigen BLAS* از این کتابخانه‌ها استفاده میکنند.

نکته این است که در بعضی موارد احتمالاً برای راحتی کدنویسی، از *wrapper*های زبان *c/c++* که برای دسترسی به زبان *FORTRAN* انجما شده، استفاده میکنند.

نکته این است که ماتریکس‌ها در زبان $c/c++$ و یا جاوا، دیفالت به صورت $row - major$ ذخیره میشوند اما *FORTRAN* با ماتریکس‌های $column - major$ کار میکند. پس از همان اول بهتر است ماتریکس‌های خود را به فرمت $column - major$ ذخیره کنیم تا از *overhead* منطبق شدن فرمت ماتریکس‌ها جلوگیری کنیم. حتی این موضوع در نحوه خواندن و تعداد *IO*‌های لازم برای آوردن بخش مربوطه از ماتریکس به مموری (مثلا بلاک مربوطه) نیز مهم است. در صورت منطبق نبودن فرمت، ممکن است به تعداد بسیار زیادی *IO* داشته باشیم.

۴

اسلاید ۲ از صفحه ۱۷ [۱]

یک ماتریکس با ابعاد $n \times m$ زمان *sparse* است که دو تا شرط داشته باشد.

۱. تعداد عناصر غیر صفر بسیار کمتر از ابعاد باشد، مثلا یک اردر کمتر باشد. $n_{nz} \ll nm$

۲. یک الگوی *sparsity* خاص و مشخصی داشته باشد. (مثلا اینکه فقط عناصر قطری غیر صفر باشند)

در این حالت میتوان از بهینه سازی استفاده کرد. مثلا ماتریس *sparse* را میتوان با یک ماتریکس $3 \times n_{nz}$ عنصره نشان داد.

استفاده از روش *factorization* چولسکی: گاهی اوقات لازم است که یک ماتریکس را *factorize* کنیم به $A = L \times L^T$. حالا اگر A یک ماتریکس اسپارس باشد، دوست داریم که L نیز اسپارس باشد. اما این مسئله $NP - hard$ است. یعنی ممکنه L ماتریکس *dense* بشه اگه بدون جایگشت انجام بدیم و مثلا با جایگشت *sparse* میشد. پس ممکنه ماتریکس *dense* درست بشه و این خوب نیست. پس همون الگوی *sparsity* ماتریکس اول هم است برای محاسبات معادلات آینده تا ماتریکس *dense* درست نشود.

درمورد *CPU* و *GPU* بنظرم که استفاده از *CPU* برای ماتریکس‌های *sparse* که میتوانند عناصر غیرصفر در الگوهای خاص داشته باشند، بهتر است. در صورتی که ماتریکس‌های *dense* بهتر است که در *GPU* محاسبه شوند چون منظم هستند و قابلیت موازی سازی زیاد و *utilization* زیادی میتوان از *GPU* گرفت. از طرفی یک سری از الگوریتم‌های پیچیده راحت ترست که روی *CPU* اجرا شود. مثل الگوریتم *factorization* چولسکی برای توابع اسپارس.

کلا چون *GPU* از چند تا *thread* استفاده میکند، ممکن است در ماتریکس‌های اسپارس، یک *thread* کارش زودتر تموم بشه از اون یکی چون مقادیر غیر صفر بیشتری دارد، و بیکار میماند در ادامه کار.

از طرفی چون عناصر ماتریکس در ماتریکس‌های اسپارس فشرده ذخیره میشود، دسترسی به حافظه در *CPU* برایش سریع تر است تا *GPU* چون در *GPU* راخت تر است یک بلاک از حافظه بخواند و ممکن است سر ایندکس‌های نامنظم عناصر غیر صفر، دسترسی های *IO* خیلی زیاد شود. و *CPU* در این موارد دسترسی زیاد بهتر است.

این نکات مطرح شده برای مقایسه *GPU* و *CPU* را از جمینای پرسیدم که مقایسه کند. [۲]

در ضمن استاد هم گفتند برای ماتریکس های کوچک لازم نیست سراغ ذخیره سازی در *GPU* برویم چون *overhead* زیادی دارد.

۵

اسلاید ۶ صفحه ۱۱ [۱]

نسل اول رجیستر های *SIMD* اینتل *MMX* ها بودند. که ۶۴ بیت حافظه داشتند. نسل بعد *XMM* ها بودند. که یجورایی ۱۶ تا رجیستر ۱۲۸ بیتی بود. البته تا ۳۲ تا رجیستر هم در نسل های آخر *AVX - 512* رسید.

نسل بعدی *YMM* ها بود. که رجیستر های ۲۵۶ بیتی بودند. اول ۱۶ تا رجیستر از این نوع کنار هم بودند و در نسل *AVX - 512* به ۳۲ تا رجیستر هم میتونه اکستند بشه.

نسل آخر *ZMM* ها بودند. این رجیستر ها ۵۱۲ بیت حافظه دارند. مثلاً ۳۲ تا رجیستر از این نوع کنار هم میذارند اسلاید ۶ صفحه ۱۶ [۱]

data alignment: اینکه با آدرس هایی از حافظه کار بکنیم که مضربی از ۱۶، ۳۲، ۶۴ و غیره هستند یعنی توان های دو یجورایی. چون که مثلاً دیتا هایی که ما در این رجیستر ها ذخیره میکنیم، مثلاً اعداد فلوت یا اینتیجر، میتونن ۱۶ یا ۳۲ یا ۶۴ بیت باشند. پس بهتره از ابتدای همین مضرب ها در حافظه ذخیره بشه. (اگه مثلاً وسطش ذخیره بشه توی مثلاً دو تا ۳۲ بیت متوالی هست، هر دو بخش پشت سر هم رو باید بخونیم (دو دسترسی شاید) و با هم *merge* کنیم و پردازش مرتب کردنش را داریم و کلاً *overhead* الکی محاسباتی داره) و کارایی را کاهش میده

branching: از اونجایی که *SIMD* است، میتواند تک *instruction* را روی دیتا های مختلف انجام دهد، پس بهتر است کد ما *jump* نداشته باشد و یک روند اجرای یکسان خطی داشته باشد و سر *if* و *else* دودوستگی دستوری و محاسباتی رخ ندهد. چون یک کد است، شاید بیاید و همه دیتا ها را برای تمام حالت ها در نظر بگیرد و این خودش *overhead* محاسباتی و مموری زیادی و نامفید میگیرد. نمیتونه کد رو دو تیکه کنه که و داده ها رو تقسیم کنه و برای یه عده، یه دستور و برای یه عده دیگه، دستور دیگه ای اجرا کنه.

۶

اسلاید ۶ [۱]

جواب نهایی برابر خواهد بود با:

a_{33} a_{23} a_{13} b_{31} b_{32} b_{33}
 a_{32} a_{22} a_{12} b_{21} b_{22} b_{23}
 a_{31} a_{21} a_{11} b_{11} b_{12} b_{13}

a_{ij} , b_{ij}

$a_{11}=1$ $b_{11}=2$

$a_{12}=0$ $b_{21}=-2$
 $a_{21}=-1$ $b_{12}=1$

$a_{13}=2$ $b_{31}=1$
 $a_{22}=3$ $b_{22}=3$
 $a_{31}=2$ $b_{13}=0$

$a_{23}=1$ $b_{32}=4$
 $a_{32}=-1$ $b_{23}=0$

$a_{33}=0$ $b_{33}=-1$

شكل ١: systolic example

۷

اسلاید ۹ صفحه ۵ [۱] و این لینک یوتیوب [۳]

این الگوریتم از دو فاز تشکیل شده است:

فاز اول: *share - reduce* یا *scatter - reduce* یا همان مرحله *summing*:

در این مرحله، با توجه به ویدیو، نود ها *chunk* های داده را به گونه ای بین هم رد و بدل میکنند که در نهایت، هر کدام از آن ها در نهایت یک داده کامل متفاوت *reduce* شده در اختیار دارند. مثلاً نود یک، داده شماره ۱ را به طور کامل دارد، نود دو داده شماره ۲ را و الی آخر [۴]

درواقع هر نود در هر مرحله، یک *chunk* داده را به نود بعدی میفرستد، و از نود قبلی هم یک چانگ دیتا دریافت میکند.

هر نود پس از دریافت چانگ جدید دیتا از نود قبلی، آن را طبق تابع موجود مثلاً *add* با حاصل چانگ های قبلیش جمع میکند. این کار را در $N - 1$ مرحله انجام میشود تا چانگ های آن داده خاص در $N - 1$ نود دیگر را بگیرد. پس برای این مرحله *scatter - reduce*، باید هر رنک یا نود یک *mpi - send* به رنک بعدی $(rank + 1) \% N$ در دور بزند و یک *mpi - recv* از رنک قبلی $(rank - 1 + N) \% N$ داشته باشد. میتوان هر دوی آن ها را در دستور *mpi - sendrecv* با هم ادغام کرد و انجام داد. [۴]

فاز دوم: *share*: یا همان *All - gather*:

در این فاز، چون هر نود، یک نوع داده کامل را دارد، نود ها این نوع داده های متفاوت را با *share* میکنند تا در نهایت همه نود ها، همه نوع داده ها را به صورت کامل داشته باشند.

مثل مرحله قبل، نود ها دیتا را به نود بعدی ارسال و از نود قبلی میگیرند و این کار را $N - 1$ بار انجام میدهند. تفاوت این است که به جای اینکه دیتای جدید را با دیتای موجود قبلی خودشان مثلاً جمع کنند، دیتای جدید را در اسلات مربوطه به آن نوع دیتا کپی میکنند. خودشان از قبل یک اسلات را پر کردند و حالا در این مرحله باید با گرفتن داده های دیگر نود ها، بقیه اسلات ها را پر کنند. [۴]

برای این مرحله *All - gather* هم میتوان یک *mpi - send* و *mpi - recv* را با هم ترکیب کرد و *mpi - sendrecv* استفاده کرد.

هر دو مرحله را $N - 1$ بار انجام میدهیم.

۸

۱

خب در ابتدا باید داده ها را به N نوع داده تقسیم کنیم. حجم هر نوع داده S/N بایت است. حال در مرحله اول $N - 1$ بار ارسال داشته باشیم و $reduce$ کنیم. در مرحله $All - gather$ هم باید $N - 1$ تا دیتای کاهش یافته را (با حجم S/N بایت) با هم رد و بدل کنیم. پس در نهایت داریم:

$$\frac{S \times (N - 1)}{N} + \frac{S \times (N - 1)}{N} = \frac{2 \times 8 \times 10^9 \times 7}{8} = 14GB$$

۲

طبق [۵] و اسلاید ۷ درس صفحه ۴ [۱]، برای ۱۶ باند، پهنای باند $PCIe\ gen\ 4$ برابر با $31.5GB$ است. برای سادگی فرض میکنیم زمان پردازش ها ناچیز هستند و فقط زمان ارتباط گرفتن همزمان وجود دارد. پس باید بین N تا GPU آن را تقسیم کنیم و به هر کدام، $31.5/8 = 3.93GB$ پس در نهایت داریم:

$$\frac{14\ (GB)}{3.93\ (G/s)} = 3.56\ sec$$

۳

در اسلاید ۷ صفحه ۷ [۱]، پهنای بند رفت در $5\ nvlink$ برابر 900 گیگابایت است. از طرفی تمام ورژن های $nvswitch$ تا ۸ تا GPU را ساپورت میکنند. (اسلاید ۷ صفحه ۱۴) پس مشکلی نیست. پس در نهایت داریم:

$$\frac{14\ (GB)}{900\ (G/s)} = 15.6\ ms$$

چون زمان $communication$ برابر $15.6\ ms$ شد که از زمان $computation$ که $120\ ms$ است، خیلی کمتر است، $bottleneck$ نمی شود.

اما در حالت اول که زمان $communication$ برابر با $3.56\ sec$ بود، چون خیلی بیشتر از زمان مناسبه است، گلوگاه میشود.

همانطور که دیدید، $nvlink$ به مراتب پهنای باند بیشتری نسبت به $PCIe\ 4$ بین GPU ها ایجاد میکند و $latency$ آن به مراتب کمتر است.

همچنین به خاطر توپولوژی درختی / اشتراکی، ممکن است که موقع ارتباط گرفتن به *conflict* بخوریم و از آنجایی که *nvl* بین دو *GPU* است و همه به همه لینک مستقیم دارد به نوعی، به این مشکل نمیخوریم و *conflict-free* است. چون *bottleneck* هم نشد، پس بهترین انتخاب همان *nvl* است. البته قطعا هزینه بیشتری دارد.

۹

اسلاید ۷ صفحه ۳۲ [۱]

۱

طبق صحنه ۳۵. *NCCL* برای تنسور و در نتیجه مدل های زبانی بزرگ که از آن ها تشکیل شده است، از بقیه بهتر است. فارق از سایز کلاستر و تعداد *worker* ها و از آنجایی که مدل های زبانی بزرگ فضای حالت موازی سازی بیشتری دارند، پس میتوان از *GPU* برای آن استفاده کرد. (توجه کنید که *gloo* برای *CPU* بهینه شده است و *MPI* هم برای تنسور ها و در نتیجه مدل های کوچک مناسب است و در مدل های بزرگ بلاشک نیکل) (*NCCL* در *f16* کمی بهتر است و در *f32* قطعا انتخاب بهتری است. پس در هر دو حالت *NCCL*)

۲

چون باید *CPU* استفاده کنیم ، پس امکان استفاده از *NCCL* که فقط برای *GPU* است را نداریم. البته باید توجه داشته باشیم، اگر فرمت ذخیره سازی *f16* است، قطعا تنها انتخاب *gloo* است، اما اگر *f32* باشد، طبق صفحه ۳۳، در حالتی که حجم پیام ها کوچک است، *MPI* عملکرد بهتری دارد. مخصوصا برای تنسور های سایز کوچک. (ولی خب در سایز بزرگ، *gloo* بهتر عمل میکند خب)

میتوانید در جدول ۲ یک سری مقایسه و توضیحات ببینید: [۶]

در مقایسه خود صفحه ۳۲ هم آمده که مثلا *mpi* سعی کرده برای *cpu* و *gpu* بهینه بشه ولی خب برای تنسور های کوچک جوابه، و زیاد هم *native* روی *gpu* نیست. *nccl* هم که فقط روی *gpu* هست. *gloo* بیشتر برای *cpu* بهینه شده ولی یک سری توابع از *gpu* را هم بهینه کرده.

به جز *mpi* که فقط با *f32* کار میکند، هر دوی دیگر هم *f32* هم *f16* را ساپورت میکنند.

Feature	MPI (Message Passing Interface)	Gloo	NCCL (NVIDIA Collective Comm. Library)
Primary Hardware	CPU (Can support GPU, but less native)	CPU (also supports GPU, but slower)	NVIDIA GPU (Exclusive)
Primary Strength	extremely mature; low latency; complex p2p patterns	Portable; native to PyTorch; good for sparse data	Max Bandwidth; Topology-aware (NVLink/PCIe)
Best For	HPC, Scientific Simulations, Supercomputing	Distributed CPU training, coordinating processes	Deep Learning on GPUs (Dense tensors)

شکل ۲: comparison of MPI, GLOO, NCCL

۱۰

اسلاید ۹ [۱]

طبق اطلاعات موجود در اسلاید ۹ درس چنین جدولی داریم ۳. (پی دی اف درس رو بهش دادم و از جمینای خواستم که طبق اون و اطلاعات خودش، به این سوال جواب بده) [۷]

۱۱

اسلاید ۹ صفحه ۸ [۱]

۱

درمورد $SFB = \text{sufficient factor broadcasting}$:

از آنجایی که حجم ماتریس ها در اردر n^2 است، ممکن است $overhead$ ارتباطی زیادی داشته باشد. پس سعی میکنند آن را به دو تا وکتور $n \times 1$ و $1 \times n$ بشکنند و این دو وکتور را جابجا کنند. (ممکن است شکسته نشود یا اگر هم شکسته شد، دقتش کمتر از ماتریس اصلی باشد) و چون هزینه ارتباطی از $2n$ خواهد شد، به صرفه میتواند باشد. مثلا ماتریس $m \times n$ به دو وکتور $m \times 1$ و $1 \times n$ شکسته شده و اردر از $m \times n$ به $m + n$ میرسد.

ویژگی / توپولوژی	۱. مسیر و نحوه تجمیع گرادیان/پارامتر	۲. الگوی ارتباطی هر نود (هزینه ارتباط)	۳. نقطه شکست واحد (SPOF)	۴. مقیاس پذیری با افزایش تعداد نودها
Centralized (متمرکز)	تجمیع تنها در یک نود مرکزی انجام می شود. مدل ها معمولاً به صورت Ensemble ترکیب می شوند.	ارتباط سلسله مراتبی است. همه نودها داده ها را به سرور مرکزی می فرستند. فشار ارتباطی روی نود مرکزی بسیار زیاد است.	بله. خرابی نود مرکزی کل سیستم را متوقف می کند.	پایین. نود مرکزی سریعاً به گلوگاه تبدیل می شود.
Tree (درختی)	تجمیع میانی انجام می شود. گرادیان ها از پایین به بالا جمع می شوند و سپس Broadcast می گردند.	هر نود فقط با والد و فرزندان خود در ارتباط است. هزینه توزیع شده اما ریشه ترافیک بالایی دارد.	بله. ریشه SPOF است و خرابی نود میانی یک زیرشاخه را قطع می کند.	متوسط/خوب. تأخیر رشد لگاریتمی دارد.
Ring (حلقه)	داده ها در یک جهت بین نودها می چرخد تا همه نودها نسخه کامل دریافت کنند.	هر نود فقط با دو همسایه صحبت می کند. پهنای باند مصرفی ثابت است.	بله. خرابی یک نود حلقه را قطع می کند مگر مکانیزم ترمیم باشد.	خوب. پهنای باند ثابت، اما تأخیر خطی با تعداد نودها افزایش می یابد.
Parameter Server (سرور پارامتر)	مدل بین چندین سرور پارامتر تقسیم می شود و Workerها گرادیان ها را به آنها ارسال می کنند.	Workers با سرورهای PS در ارتباط اند. پارامترها در حافظه مشترک جهانی نگهداری می شوند.	بله (جزئی). خرابی PS بخشی از مدل را از بین می برد.	متوسط/خوب. بار توزیع می شود ولی PS همچنان گلوگاه است.
Peer-to-Peer (نظیر به نظیر)	هیچ نود مرکزی وجود ندارد. همه نودها نسخه کامل پارامترها را دارند.	نودها می توانند مستقیم با یکدیگر ارتباط داشته باشند (Broadcast یا Gossip).	خیر. SPOF وجود ندارد.	بسیار بالا. Gossip مقیاس پذیری عالی ایجاد می کند.

شکل ۳: topology comparison

۲

در مورد *gossip learning*

در این مدل، هر نود، به نود های همسایه *update* می‌دهد فقط و همینطور این آپدیت در مراحل بعد به نود های همسایه های همسایه و بعدا بیشتر منتقل میشود. چون *update* را به کل نود های لازم نمیدهد، ممکن است در ابتدا کمی *diverge* کند. پس اجازه *diverge* را میدهد اما بعدا و در دراز مدت که این آپدیت ها به نود های بیشتری منتقل شد، *converge* میکند. پس سرعت همگرایی اش کمتر است اما خوب هزینه *communication* را کاهش داده و به صورت همتا به همتا با همسایه های مستقیم فقط آپدیت ها را منتقل میکند.

۱۲

اسلاید ۸ [۱]

۱

در روش *data parallel* محض، باید کل مدل را در هر *replica* قرار دهیم، اما مدل ما ۱۶۸ گیگابایت حجم دارد و حجم حافظه های *GPU* ما ۲۴ گیگابایت جا دارد که ۴ گیگابایتش برای دیتا میره و فقط ۲۰ گیگابایت برای ذخیره سازی مدل جا داره که اصلا کافی نیست. پس باید مدل را نیز پارتیشن کنیم.

۲

چون مدل ۱۶۸ گیگابایت است و هر *GPU* فقط ۲۰ گیگابایت برای مدل جا دارد، پس مدل را حداقل باید به $\lceil \frac{168}{20} \rceil = 9$ قسمت پارتیشن کرده تا در ۹ تا *GPU* قرار دهیم.

۳

در روش *FSDP = Fully Sharded Data Parallel* به جای اینکه هر *GPU* کپی کل مدل را نگه دارد، وضعیت های مدل (*States Optimizer, Gradients, Weights*) را بین تمام *GPU* های موجود تکه تکه (*Shard*) می کند.

اگر N کارت گرافیک داشته باشیم، هر کارت فقط $\frac{1}{N}$ از پارامترها را ذخیره می کند. در لحظه محاسبه (*Backward/Forward*) برای هر لایه، آن *GPU* تکه های مورد نیاز آن لایه خاص را از سایر *GPU* ها می گیرد، محاسبه را انجام می دهد و سپس بلافاصله داده های گرفته شده را آزاد می کند تا حافظه پر نشود.

برای همین، در $FSDP$ ارتباطات سنگین تر و متراکم تر است. و شبکه مدام درگیر است و به جای یک $all - reduce$ در آخر هر آپدیت، مدام، $all - gather$ و $reduce - scatter$ انجام میدهد. [۸]

۱۳

اسلاید ۷ صفحه ۲۸ [۱]

پروتکل $RDMA$ از مفهومی به اسم $zero - copy$ استفاده میکند. به این شکل که لازم نیست دیتا بین بافر های لایه های مختلف شبکه، مثل $Application$ و $kernel$ و ... تا $hardware$ جابجا شود و از طریق لایه فیزیکی به مقصد برسد و دوباره در مقصد همین مسیر را از پایین به بالا طی کند. یک لینک ارتباطی مستقیم بین $application$ مبدا و مقصد ایجاد میکند و $kernel - bypass$ کرده و درواقع دیتا را از همان لینک مستقیم جابجا بین اپلیکیشن مبدا و مقصد جابجا میکند. اینگونه به کپی کردن مداوم در بافر ها نیازی نداریم و مستقیم دیتا را منتقل میکنیم به مقصد. درواقع یک میانبر میزند بین دو تا پروسس مبدا و مقصد.

پس سر بار ارتباطی و کپی کردن در حافظه های بافر های متعدد را به شدت کاهش میدهد. برای $model parallelization$ و اینکه ارتباط سریعی بین GPU ها برقرار باشه و پهنای باند و سرعت بالا و تاخیر کم داشته باشند، به $RDMA$ و $IB switch$ ها نیاز داریم.

این پروتکل در $InfinityBand$ که از کارت شبکه HCA استفاده میکند، کاربرد دارد. (جهت یادآوری، چون $Ethernet$ کند است و تاخیر زیادی دارد، از $InfinityBand$ در لایه دیتالینک استفاده میکنیم. که مخصوص $inter - process communication$ است.)

پس برای محاسبات بزرگ و پیچیده که به پرفورمنس ارتباطی خوب نیاز دارند، کاربرد دارد.

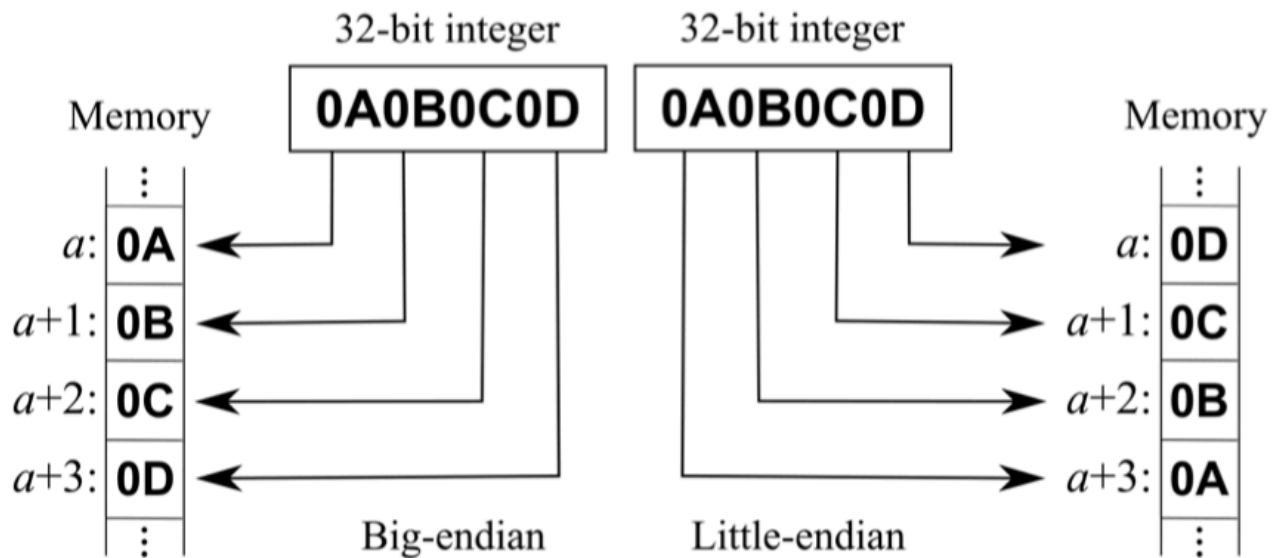
۱۴

اسلاید ۹ صفحه ۱۰ [۱]

۱

همانطور که میبینید ۴، در روش $little - endian$ بیت های کم ارزش تر، در آدرس های پایین تر ذخیره میشوند و بیت های با ارزش تر در آدرس های بالاتر. در صورتی که در $big - endian$ بیت های با ارزش تر در آدرس های پایین تر حافظه ذخیره می شوند.

مثال شکل، مثال یک عدد ۳۲ بیتی است که به صورت هگز $0A0B0C0D$ است. نحوه ذخیره سازی در شکل مشخص است.



شکل ۴: big and little endian

۲

پروتکل‌های *serialization* که مثلاً در *object*‌های جاوا از آن استفاده شده، این موضوع را از دید کاربر *abstract* کرده و خودش با توجه به سیستم در حال استفاده و نحوه ذخیره سازی در آن، این موضوع را هندل میکند و تبدیلات و انطباق را در صورت لزوم انجام میدهد.

۱۵

اسلاید ۶ صفحه ۵۳ [۱]

همانطور که در صفحه ۵۳ درس مشخص است، میتوان در جدول ۵ مقایسه آن‌ها را دید. (مانتیساً همان تعداد بیت اعشار است و دقت را نشان میدهد و *exponent* رنج را نشان میدهد)

۱

مشکل اصلی $FP16$ داشتن تنها ۵ بیت برای توان (*Exponent*) است. این یعنی کوچکترین عدد مثبتی که می‌تواند نمایش دهد (بدون *denormalization*) حدود 6×10^{-5} است. در شبکه‌های عصبی عمیق، گرادینت‌ها معمولاً اعداد بسیار کوچکی هستند (مثلاً 10^{-7} یا کمتر).

ویژگی	FP32	FP16	bfloat16 (BF16)
طول کل	۳۲ بیت	۱۶ بیت	۱۶ بیت
ساختار (Exponent / Mantissa)	۸ بیت / ۲۳ بیت	۵ بیت / ۱۰ بیت	۸ بیت / ۷ بیت
بازه دینامیکی (Dynamic Range)	بسیار وسیع (حدود 38×10^38 تا 38×10^{-38})	بسیار محدود (حدود 5×10^6 تا 6.55×10^{-4})	وسیع (تقریباً مشابه FP32)
دقت (Precision)	بسیار بالا	متوسط	پایین (کمتر از FP16)
نتیجه‌گیری	استاندارد طلایی اما پرهزینه	مستعد Underflow / Overflow	بهترین توازن برای Deep Learning

شکل ۵

پدیده *Underflow*: وقتی گرادیان محاسبه شده کوچکتر از حداقل قابل نمایش در *FP16* باشد، مقدار آن صفر مطلق در نظر گرفته می‌شود. این یعنی اطلاعات گرادیان کاملاً از دست می‌رود. هنگام جمع زدن گرادیان‌های کوچک با یک مقدار بزرگتر، به دلیل دقت پایین (*Mantissa* کم)، مقدار کوچک ممکن است محو شود و در عمل جمع تاثیری نگذارد.

اثر بر *Vanishing Gradient*: پدیده محو شدن گرادیان (*Gradient Vanishing*) زمانی رخ می‌دهد که گرادیان‌ها در حین انتشار به عقب (*Backpropagation*) به تدریج کوچک می‌شوند. در واقع به نوعی گرادیان‌ها به صورت زنجیره ای در هم ضرب میشوند و همین موضوع میتواند اعداد کوچکتری تولید کند. در حالت *FP32*، این اعداد کوچک همچنان وجود دارند و شبکه (هرچند کند) یاد می‌گیرد. اما در *FP16*، به محض اینکه گرادیان از آستانه دقت پایین‌تر بیاید، ناگهان صفر می‌شود. این باعث می‌شود که یادگیری در لایه‌های ابتدایی شبکه کاملاً متوقف شود و همگرایی مدل شکست بخورد.

پس چون بازه محدود است، مشکل *overflow/underflow* داریم.

اگر هم یک موقع یک عدد کوچک را با یک عدد دیگر جمع بزنیم، ممکن است گرد شود و اصلاً اثر آن حذف شود. [۹]

۲

تکنیک *Scaling Loss* یک راهکار ساده اما حیاتی در آموزش *Precision Mixed* است. در این روش، مقدار *Loss* محاسبه شده در انتهای *Pass Forward*، در یک عدد بزرگ (*Factor Scale*)، مثلاً 10^{24} یا بیشتر، ضرب می‌شود.

از طرفی چون مشتق، یک عمل خطی است، ضرب در آن مشکلی ایجاد نمیکند. یعنی الان میتوانیم ضرب کنیم و بعدا دوباره به همان عدد تقسیم کنیم. مراحل انجام آن به این شکل است:

۱. *Shift* کردن مقادیر: با این کار، گرادیان‌های بسیار کوچک که در محدوده خطر (*Underflow*) بودند، بزرگ می‌شوند و به محدوده امن (*Range Representable*) فرمت *FP16* منتقل می‌شوند.
۲. حفظ اطلاعات: حالا که گرادیان‌ها صفر نشده‌اند، عملیات *Backpropagation* انجام می‌شود.
۳. *Unscaling*: قبل از اینکه وزن‌های مدل آپدیت شوند (توسط *Optimizer*)، گرادیان‌ها دوباره بر همان عدد *S* تقسیم می‌شوند تا به مقدار واقعی خود برگردند و گام آپدیت صحیح برداشته شود.

۱۶

اسلاید ۸ [۱]

۱

ابتدا سائز نمونه را پیدا میکنیم:

$$\frac{1.5 \times 10^9 \times 200 \times 32}{8} = 1200 \times 10^9 \text{ GB} = 1.2 \text{ TB}$$

پس نمیتوان این سائز دیتا را در یک *GPU* با ظرفیت ۱۲ گیگابایت جا داد.

۲

به جای آوردن کامل *batch* در حافظه، به صورت *mini-batch* دیتا‌ها را می‌آوریم یعنی تیکه تیکه دیتا‌ها را می‌آوریم و برای هر کدام یک *iteration* و یا مثلا داده‌ها را تک تک به مدل میدهد تا آپدیت کند به صورت *stochastic*. مثلا برای *update* کردن از روش‌های *mini-batch gradient decent* یا روش *stochastic gradient decent* مثلا میتوان استفاده کرد.

۳

میتوانید در شکل ۶ مقایسه آنها را ببینید. [۱۰]. علاوه بر آن، *utilization* حافظه در *mini-batch* بهتر از *stochastic* است. در شکل ۷ که اسلاید ۸ صفحه ۴ [۱] است هم برخی از مقایسه‌ها را می‌بینید. مثلا با توجه به اینکه سائز نمونه‌ها در *mini-batch* کمتر از *batch* و در *stochastic* کمتر از *minibatch* است، سرعت *iteration* و هر آپدیت برای *stochastic* بیشتر از *mini-batch* و آن هم بیشتر از *batch* است، اما چون لازم است که *iteration*

ویژگی	Stochastic (Batch Size = 1)	Mini-batch (Batch Size = 32~512)
سرعت پردازش هر Epoch	بسیار کند (عدم استفاده از توان GPU)	سریع (استفاده از پردازش موازی GPU)
فشار روی دیسک (I/O)	بسیار زیاد (گلوگاه شدید)	بهینه و مدیریت شده
پایداری همگرایی	پایین (نوسان شدید)	بالا (حرکت نرم‌تر به سمت هدف)
دقت نهایی	خوب (اما نوسانی)	عالی (استاندارد شبکه‌های عمیق)
ریسک گیر افتادن در مینیمم محلی	کم (به دلیل نویز بالا می‌پرد)	متوسط (قابل کنترل با تنظیم Batch Size)

شکل ۶: minibatch vs. stochastic

های زیادی انجام شود تا یک *epoch* کامل دیتا دیده شود، پس تعداد *iteration* ها در یک *epoch* در استوکستیک خیلی بیشتر از مینی بچ و آن هم بیشتر از بچ است که فرمول های تعداد *iteration* در هر *epoch* در شکل ۷ آمده. پس برای همین *overhead* ارتباطی در استوکستیک خیلی زیاد است و *communication* گلوگاه میشود در یک ایپاک. بعد *mini - batch* و کمترین سربار ارتباطی هم در یک ایپاک برای همان *batch* است.

از آنجایی که مدل آپدیت های زود به زود تری در استوکستیک دریافت میکند، سریع تر میتواند *train* شود و همچنین به حافظه کمتری هم نیاز دارد اما در مقابل بخاطر نوسانی حرکت کردن به سمت نقطه مینیمم گلوبال، شاید دقیق نباشد و یک الگوی منظم در حرکت به سمت مینیموم ندارد. البته همین میتواند کمک کند که از *local minima* خارج شود ولی خب خیلی تقریبی عمل میکند.

mini-batch متعادل است و آپدیت های کمتری نسبت به آن میگیرد، سربار ارتباطی متوسطی دارد، از *local minima* به خاطر نوسان های متوسط میتواند خارج شود، و منظم تر به نقطه مینیمم گلوبال *converge* میکند. همچنین میتوان گفت بهره وری *GPU* در *mini - batch* چون بالاخره چند تا نمونه داریم و میتوانیم در ترد های مختلف موازی سازی کنیم، بهتر از تک نمونه در *stochastic* میتواند باشد.

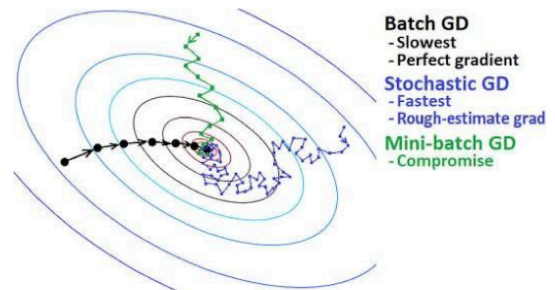
□ Batching allows the gradient to be calculated for part of the data during each **iteration** of training.

➤ Benefits:

- Requires less memory
- Networks train faster. Why?
 - Due to more frequent updating of network parameters

➤ Cons:

- Gradient calculation would be less accurate



Source: <https://sweta-nit.medium.com/batch-mini-batch-and-stochastic-gradient-descent-e9bc4cad461>

شکل ۷: comparison

مراجع

- [1] “dmls-slides,” <https://elearn.ut.ac.ir/mod/folder/>, 2025.
- [2] “gpu-vs-cpu,” <https://gemini.google.com/share/>, 2025.
- [3] “ring all reduce,” <https://www.youtube.com/watch>, 2024.
- [4] “scatter-reduce,” <https://gemini.google.com/share/>, 2025.
- [5] “pcie4,” <https://www.trentonsystems.com/en-us/resource-hub/blog/pcie-gen-4>, 2021.
- [6] “mpi gloo nccl comparison,” <https://gemini.google.com/share/>, 2025.
- [7] “topology comparison,” <https://gemini.google.com/share/>, 2025.
- [8] “fsdp,” <https://gemini.google.com/share/>, 2025.
- [9] “fp16,” <https://gemini.google.com/share/>, 2025.
- [10] “batch,” <https://gemini.google.com/share/>, 2025.